



UniDash

Integration blockers and implementation handoff

A detailed brief for a specialized AI or engineer to continue the Moodle and AMS scan/report feature.

USER GOAL

After UniDash sign-in, open the student's college portals, let the student sign in there, scan for records that changed since the last scan, and show a trustworthy change report on the dashboard. Hourly auto-login/background polling is not wanted for the current version. The user is interested in an Edge/Chrome extension to bridge the browser pages.

Prepared: 03 October 2026 (Asia/Kolkata)

Project: UniDash | Student dashboard for MITS Gwalior

Scope: Integration feasibility, current implementation, blockers, proposed extension design, handoff instructions

1. Executive summary

The desired feature is a user-triggered portal scan with a delta report, not a scheduled scraper. The app now has a sign-in-time prompt and can open the MITS portal pages, but it does not log in for the user, inspect those pages, import portal records, detect changes, or render a real scan report. This distinction must remain explicit in all UI and future AI work.

Main technical constraint

UniDash runs at localhost:3000 while the portals run on separate MITS origins. Normal page JavaScript cannot read another tab's page DOM. An extension or a portal-supported API/feed is required. The current browser prompt only opens tabs.

The most practical first implementation is an explicit Edge/Chrome extension action. The user signs in directly on Moodle and AMS; after the user clicks Scan, a narrowly scoped extension reads only supported, visible, user-owned records from the allow-listed portal tabs, normalizes the records, and submits them to an authenticated UniDash ingestion endpoint. The server compares stable record IDs and content hashes to the previous successful scan, records change history, and returns a report for the dashboard.

Moodle has two plausible routes: an official read-only Web Services token (currently unavailable to the owner in the UI) or a dynamic Moodle calendar feed (observed in the portal, but no private feed URL has been generated or ingested). AMS exposes user-specific attendance pages, but no supported API/export was found in the inspected Developer/Downloads UI. AMS needs an approved source route or a permission-reviewed browser-extension adapter before it can be called a reliable integration.

Google sign-in to UniDash is not itself portal authorization. Reuse of the same Gmail identity is possible only if the MITS portals are configured for that Google identity provider. Even then, the portal performs its own SSO; UniDash must not pass its Google login token to the portals or collect the user's Gmail password.

Decision snapshot

Area	Current state	Meaning
Post-login prompt	Implemented in local frontend	Appears when integration status loads and one or both sources are disconnected; opens only needed MITS portal URLs.
Moodle token	Verification endpoint only	Validates core_webservice_get_site_info and encrypts an accepted token; no content import.
Moodle calendar feed	Observed, not connected	Dynamic export URL appears available; no URL generated, stored, or fetched.
AMS	No source connector	UI pages showed attendance; no official API/export route established.
Scan/delta/report	Not implemented	No manual scan action, scan-run history, normalized portal import, or dashboard report.
Background schedule	Not requested	Hourly scanning was explicitly set aside by the owner.

2. User request and product behavior

Owner context: 1st-semester B.Tech CSE at Madhav Institute of Technology & Science (MITS), Gwalior, Madhya Pradesh; owner confirms CSE Section B and lab batch B3. AMS is the source of truth for attendance. Moodle is used for course-related dates/resources.

Requested behavior, stated plainly

- 1 The student signs in to UniDash using Google/Gmail.
- 2 If a portal still needs setup, UniDash prompts the student and can open Moodle/AMS.
- 3 The student signs into each official portal in the browser. The student wants this to be automatic where the same Google login is supported.

- 4 The student presses a scan action. No hourly background scanning is needed for this version.
- 5 UniDash compares what it can see now to the last successful scan and reports new, changed, and possibly removed records on the dashboard.

Recommended first-release behavior

- Ask only while a portal is disconnected or needs reauthorization; provide Later. Do not block dashboard use.
- Opening a portal is not a successful connection and is not a scan. Show separate states: Portal opened, Signed in (if detectable), Scan ready, Scanned at, and Needs attention.
- Keep scans manual and user initiated. Add an explicit Scan now action only for sources with a configured and verified read path.
- Show a per-source report: scan start/end, last successful scan, new count, changed count, unchanged count, unavailable/error count, and links to records. Distinguish 'no changes' from 'scan failed' and 'source not supported'.
- Do not mark records removed merely because a page was not visited. Absence is meaningful only after the scanner proves that a complete source listing was read successfully.

No hourly polling

Owner explicitly chose the prompt + manual scan version and set aside hourly auto-login/scanning. Do not add a worker schedule for these integrations without a later product decision.

Google login interpretation

The owner expects the Gmail used for UniDash to sign into AMS and Moodle automatically. Project feasibility notes say the owner reports Google sign-in works for both portals, but the exact MITS SSO provider, account mapping, and whether a fresh UniDash login can be reused have not been validated. Treat portal SSO as an assumption to confirm, not a capability already implemented. The browser may already have an authorized Google session; a portal's own 'Sign in with Google' flow may then be quicker, but it is still the portal's separate authentication flow.

3. Current implementation: verified facts

Canonical project root observed during this handoff is the repeated-folder path below. A wrapper directory exists one level above. The running Next.js process on port 3000 was observed to use the nested project root. Start source inspection and changes from this nested root, not the outer wrapper.

`C:\Users\LENOVO\Downloads\code-sandbox-light-12d7c413-b872-4b60-805e-286d492e421f-main\code-sandbox-light-12d7c4`

No .git directory was found at the nested project root during this inspection; do not assume a branch, commit hash, or that git status/diff will work. The root .env exists. Never read it into logs, the report, or chat; inspect variable names via .env.example and keep all values private.

Portal prompt and navigation

- Maintained UI source: js/app.js, index.html, css/v2.css; deployable public copies are under apps/web/public. scripts/sync-frontend.mjs copies root static files into the Next app.
- The desktop rail now includes an Integrations link to #more. Moodle/AMS status chips route to the same page.
- On app load, the client calls GET /api/v1/integrations. If the call succeeds and any integration is not connected, the prompt offers Open portals and Later. The prompt state is in memory for that app page.
- Open portals launches fixed MITS login URLs in separate tabs only for sources not marked connected, then routes UniDash to #more. A browser may still block pop-ups; use direct links as fallback.
- This UI does not detect that the student completed sign-in on another tab. It does not scan the pages. The integration panel copy now says data scanning/import is unavailable until a supported source is configured.

Moodle connection foundation

- GET /api/v1/integrations returns the signed-in user's AMS/MOODLE connection status; secrets are not returned.
- POST /api/v1/integrations/moodle accepts a Moodle web-service token, sends only the fixed MITS Moodle host the core_webservice_get_site_info function request, and rejects invalid/site-mismatched responses.
- On successful validation, the token is encrypted using AES-256-GCM and HKDF-derived user/integration-specific key material based on VAULT_MASTER_KEY. Token is not logged or returned. DELETE disconnects and clears it.
- Token validation sets status READY but deliberately leaves lastSuccessAt null. READY means connection verified, not records synchronized.
- The owner reports the Security keys/token option is absent in their Moodle account. No live token validation was performed.

Backend and dashboard gap

- The events table has source/sourceRef fields and a unique user+source+sourceRef key; event history and event source tables exist. These are useful building blocks for idempotent deltas.
- The feed_items table and GET/PATCH feed APIs exist, including dedupe keys and read/pin/archive state. The current frontend initializes feed from demo data and does not fetch /api/v1/feed.
- The frontend initializes attendance from js/data.js demo data. It does not currently fetch the live attendance summary API for the Radar display. Do not interpret current sample attendance cards or feed entries as scanned AMS/Moodle results.
- No scan_runs/sync_runs table, manual scan API, extension pairing flow, extension package, Moodle ICS importer, AMS parser, or scan report UI was found in the current code.

4. Source-by-source feasibility and blockers

Source	Observed facts	Blocking problems	Recommended first route
Moodle	Signed-in dashboard/calendar shows course-linked events. Calendar export offers a dynamic URL for events, courses, groups, or personal events. Owner reports Google login works and no Security keys page is visible.	No live token provided; token UI absent per owner. No URL generated/imported. Current server accepts only a web-service token. No content import or delta report.	Prefer scoped read-only web service if MITS admin enables the required user capability/functions. Otherwise accept a Moodle dynamic ICS URL in a dedicated secure field and implement manual pull/import. For visible course resources/grades, ICS alone is insufficient; use a narrow extension or official API.
AMS	Inspected signed-in UI exposed per-course attendance summary and dated records. Downloads showed no files; Developer page showed contact/developer info but no API documentation. Owner identifies AMS as attendance source of truth.	No supported API/export was found. A server cannot inspect another browser tab. Attendance parsing requires stable identity/date/course semantics and proof that user's records only are selected. Institute policy/permission is unconfirmed.	Ask MITS admin for documented read-only API/CSV/export. If unavailable and permitted, build an explicit local extension adapter for visible personal attendance pages with user click and sanitized fixtures. Do not use a server-side browser-cookie scraper.
Google / Gmail	UniDash sign-in is Google OAuth, invite-only. Owner wants that Gmail to be used for portals.	UniDash Google OAuth token is for UniDash's client/audience and is not a portal login credential. MITS Google SSO configuration is not confirmed by code.	Verify each portal offers the same institutional Google SSO and user account. Let the portal perform its own interactive SSO in the browser; never pass UniDash's token or ask for Gmail password.

Moodle administrator dependency

Moodle's official administrator documentation says a user token depends on site Web Services/protocol/service configuration and the user's permissions. The Moodle documentation states that moodle/webservice:createtoken is required for a user to create a security key, and administrators can define custom external services with only selected functions. This is general Moodle behavior, not proof that the MITS instance has enabled a particular route. The most secure API option is an MITS-approved, user-scoped, read-only service with only needed functions and an expiry/IP policy where available.

Calendar feed sensitivity

The feed URL may contain a private authentication token. It has not been generated in the user's Moodle account. Do not click Get calendar URL on the owner's behalf or expose the resulting link in a prompt/log/report. If the user manually generates one after the importer exists, they should paste it only into UniDash's HTTPS credential field, not chat or repository. Encrypt it at rest, redact all query values from logs, and provide revoke/replace/disconnect behavior. A feed should be treated like a password even if it is read-only.

5. Why a browser extension is the right bridge

The user has proposed an extension similar to the browser extension used to connect Edge to this assistant. That is a reasonable architecture for a user-initiated scan if MITS permits this kind of read-only access. A normal UniDash tab cannot read the DOM of Moodle/AMS tabs because they are different origins. A browser extension is granted narrowly declared host access and can run isolated content scripts on only the portal pages.

Suggested user experience

- 1 User signs into UniDash. The app checks integration link status and prompts only when setup is incomplete.
- 2 User clicks Connect/Install helper. If extension is not installed, UniDash gives a simple local install guide. Avoid broad enterprise permissions.
- 3 User clicks Open portals. The extension/UniDash opens only fixed Moodle and AMS origins. User logs in directly; where Google SSO is enabled, the portal may use the browser's existing Google session through its own flow.
- 4 User clicks Scan now in UniDash. Extension identifies supported portal tabs, checks page origin and login state, and reads only explicitly mapped page fields for the signed-in student's own records.
- 5 Extension normalizes records, removes personal data not needed by UniDash, computes stable keys/hashes, and sends a bounded payload through a short-lived paired connection.
- 6 Server upserts source records, records new/changed items, appends history/feed entries and scan-run summary, then refreshes the dashboard. Errors are per-source and do not turn into 'no changes'.

Permission and transport design

- Manifest V3; exact host allow-list only: moodle.mitsweb.in and ams.mitsgwalior.in. Add the localhost app origin for development only; add the final production origin only after deployment is selected.
- Use content scripts in the isolated world and run only after explicit Scan now. Prefer optional host permissions requested on connect/scan. Do not request cookies, webRequest, debugger, all-URLs, or Google account access.
- Do not collect passwords, Google OAuth tokens, portal cookies, session IDs, hidden authentication fields, or full page HTML. No background surveillance and no automatic hourly login.
- Pair extension to the signed-in UniDash user with a random one-time code, short expiry, rate limit, server-side binding, and replay protection. The project specification mentions pairing-code/session routes, but those routes/helper are not currently implemented.
- Constrain message passing to the exact UniDash origin and a known extension ID. Validate every source record on the server and ignore arbitrary client-supplied URLs.
- Raw scan content should stay in memory and be discarded after normalized extraction. Persist normalized fields, stable references, minimal hashes, and audit metadata only. Define explicit deletion/retention controls.

Extension does not magically authenticate

The extension can read pages only after the user has signed in on those portals and granted its stated host access. It does not obtain access because UniDash used Google login, and it must not copy or export browser cookies.

When an official API/feed is better

Prefer a documented official API or calendar feed over DOM extraction whenever it covers the desired data. APIs have stable IDs and structured updates. A Moodle ICS feed can support course dates/deadlines but will not automatically cover all resources, grades, submissions, or course content. AMS attendance still needs its own approved structured source or a permission-reviewed page adapter.

6. Correct change-detection and dashboard report model

The user asks for 'what is new from last scan'. This requires a persisted baseline and identity rules, not only opening pages. Build source records and scan-run reports so repeated scans are idempotent and changes are auditable.

Record lifecycle	Required logic	Dashboard result
New	Stable external key appears for first time. Save firstSeenAt, source, normalized fields, last payload hash, and source URL only when safe.	New count; title, course, date; first-seen timestamp.
Changed	Known stable key is present but important normalized fields differ. Compare deterministic hash; record field-level before/after history.	Changed count; show old -> new due date/title/status without guessing why.
Unchanged	Known key and relevant fields match last complete scan.	Unchanged count only; do not create duplicate feed noise.
Missing	Previously known item absent from a proven complete listing. Do not equate a partial page/session/error with deletion.	Show 'no longer listed' only after complete coverage is verified; otherwise leave unchanged and report partial scan.
Failed/partial	Login expired, page shape changed, permission blocked, page did not load, only some courses scanned, or no recognized records.	Clearly show partial/error and last successful scan. Never report 0 changes as success.

Data keys and diff rules

- Moodle: prefer the Moodle event/activity/course stable ID or ICS UID. Fallback key should include course stable ID, normalized title, and source date only if there is no native ID; document collision risks.
- AMS: prefer portal record IDs. If none, use course code + class date + period/slot as a candidate, but confirm the portal's meaning and correction behavior before upsert.
- Store a canonical field list per connector and hash only those fields. Whitespace/order/DOM changes must not appear as business changes.
- Dates must be normalized to UTC and rendered in Asia/Kolkata. Preserve original date/timezone or all-day semantics where source provides them.
- Keep user overrides/locked fields distinct from source values. A scan should not silently overwrite a user edit.
- For attendance, model class meeting identity and present/absent/late/leave/cancelled state; calculate percentage from imported class rows only after course/term mapping and denominator policy are verified.
- A course label 'B3' or Section B is context, not enough to infer a class row. Keep course mappings explicit and owner-confirmed.

Suggested scan report contract

```
{
  "runId": "uuid",
  "source": "MOODLE | AMS",
  "status": "SUCCESS | PARTIAL | NEEDS_REAUTH | FAILED",
  "startedAt": "ISO-8601 UTC",
  "completedAt": "ISO-8601 UTC",
  "lastSuccessfulScanAt": "ISO-8601 UTC or null",
  "counts": {"new": 0, "changed": 0, "unchanged": 0, "missing": 0},
  "warnings": [],
  "changedItems": [{"stableRef": "opaque-id", "title": "safe title", "change": "DUE_DATE_CHANGED"}]
}
```

7. Blocking issues, severity, and impact

Priority	Problem	Impact	Resolution needed
P0	No browser extension/helper exists; no scan endpoint accepts extracted records.	The prompt opens portal URLs only. The requested scan cannot execute.	Implement MV3 helper and paired, user-scoped ingestion API, or choose official API/feed for all desired data.
P0	Cross-origin boundary blocks the UniDash page from reading portal tabs.	Direct in-page JavaScript/iframe approach will fail or be blocked.	Use an extension with narrow host permissions or a supported API/feed. Do not weaken browser security.
P0	AMS source method and permission are unverified.	Attendance cannot be imported safely or reliably; no real attendance report can be claimed.	Obtain MITS-approved API/export or permission for a limited visible-page extension adapter; inspect sanitized personal-record fixtures.
P0	Moodle API token is unavailable to the owner; no calendar feed has been connected.	Moodle connection validator cannot be completed with a token; there is no existing fallback importer.	Ask MITS admin for limited read-only Web Services, or implement protected calendar feed input/import and let owner privately connect it.
P0	Google identity and MITS SSO are conflated in the product request.	UniDash login cannot grant portal permissions by itself; automatic silent portal login is not established.	Verify MITS Google SSO and account linkage separately; allow portal's own sign-in flow.
P1	Dashboard still contains demo feed and demo attendance values.	A dashboard can look live while showing sample values; high risk of false academic decisions.	Replace demo attendance/feed with live APIs or visually label preview data until imports exist.
P1	No scan-run/delta persistence/report UI.	Cannot calculate changes since prior successful scan or show reliable last-scanned state.	Add scan-run records, source stable IDs/hashes/history, and dashboard report card/API.
P1	Feed API exists but is not wired into client dashboard.	Connector-produced feed entries will not appear in current Radar feed presentation.	Fetch GET /api/v1/feed with auth/cursor; map schemas and replace demo feed on success.
P1	Portal page layouts/selectors and field semantics lack sanitized fixtures.	DOM changes can silently mis-parse dates/statuses or miss records.	Collect owner-redacted screenshots/HTML and define robust selectors/mappers with a visible partial-scan guard.
P2	Extension distribution, trusted origin, and local/production connection are undecided.	Pairing/configuration may not work outside localhost or may request too much permission.	Set local unpacked workflow first; choose production domain later; keep exact origin allow-list.
P2	No browser smoke test of the recent prompt.	Popup blocking, integration modal, keyboard/accessibility, and Edge tab behavior are not confirmed.	Owner verifies the local signed-in flow in Edge after build/restart.

Trust and safety failure modes

- False 'no changes' after an expired login or selector break.
- Duplicate events after every scan because no stable source reference was used.
- Changed/removed report based only on a subset of courses/pages.
- A calendar feed URL, token, OAuth secret, cookie, or session identifier leaks through logs, PDF, chat, or error message.
- Attendance percentages derive from timetable assumptions rather than AMS records.
- Other students' names, class roster data, or instructor-only pages are unintentionally captured.
- Source scan overwrites user edits or marks completed work incorrectly.

8. Prioritized implementation plan for the next AI

The plan below follows the owner's manual-scan preference and keeps the first data slice small. Implement one source at a time. Moodle deadlines are the likely first slice through official ICS if allowed; AMS attendance is a separate slice after access and policy evidence.

Phase A - establish the extension bridge

- 1 Create a local MV3 extension directory (for example, tools/unidash-browser-extension) with exact MITS portal host matches and no cookie permission.
- 2 Add an extension toolbar action and a UniDash 'Pair extension' screen. Generate a short-lived one-time pairing code server-side for the signed-in account, store only a hash, expire quickly, rate-limit, and invalidate after use.
- 3 Use exact allowed app origins for extension messaging. Verify sender extension ID, origin, source kind, payload schema, per-record limits, run ID, and idempotency. Do not accept user-provided URLs.
- 4 Add a Scan now action that queries only open allow-listed tabs, asks content scripts to inspect them, and returns a typed partial result if a portal is not signed in or the expected page isn't open.

Phase B - Moodle first

- 1 Choose one path: admin-approved read-only Moodle web service; or a dynamic calendar URL importer. Do not mix bearer URL or token into logs. Do not create/reveal the owner's private calendar URL on their behalf.
- 2 For ICS, implement strict `https://moodle.mitsweb.in` allow-list, redirects disabled, short timeout, response size cap, content-type/body validation, and encrypted secret storage. Protect against SSRF and user-controlled hostnames.
- 3 Parse stable UUIDs, DTSTART/DTEND/DUE, SUMMARY, DESCRIPTION, URL, TZID, all-day events and line folding/escaping. Validate maximum count/length and reject malformed feed as PARTIAL/FAILED.
- 4 Use sourceRef/UID and payload hashes for idempotent new/changed detection; preserve manual/user-locked fields. Do not label generic events as assignment deadlines without reliable source evidence.
- 5 If using Web Services, request only specific approved read-only functions. Import a small course/calendar slice before courses/resources/grades.

Phase C - AMS after evidence

- 1 Ask MITS for the supported read-only API/export and policy for student automation. If none is offered, confirm page-level automated reading is permitted before writing an adapter.
- 2 Use only owner-visible attendance pages. Capture sanitized sample structure with IDs/emails/marks/tokens redacted and never persist full page HTML or session state.
- 3 Build adapters for course mapping and attendance rows with source keys; include explicit coverage metadata (courses/date range/rows seen). Do not infer missing classes from timetable.
- 4 Implement percentage computation only from a complete, validated set of AMS meetings for the stated term/course; surface uncertainty and last checked timestamp.

Phase D - delta report and dashboard

- Add scan_runs with user/source/status/start/end/counts/coverage/error code, and migrations through Drizzle.
- Use events/event_sources/event_history and attendance tables for imported facts. Generate feed_items only after a real difference is detected, with unique dedupe keys.
- Add GET latest scan report and GET scan history endpoints; protect user scoping, CSRF for mutations, and pagination/rate limits.
- Load live feed from GET /api/v1/feed and display latest scan status/report on Radar. Keep sample state visibly marked PREVIEW when live data is absent.

- Make user action produce results synchronously if small; otherwise return a job ID and poll status. Never claim scan success until all required pages/data were actually parsed.

9. Acceptance criteria and implementation checks

Check	Pass condition
Prompt	After successful UniDash sign-in and status fetch, shows only when required. Later dismisses for that app load. Browser failure still leaves direct portal links on Integrations.
SSO	UniDash Google login remains separate. Portal login happens on portal origin. No Gmail password, OAuth token, cookie, or session ID crosses into UniDash/extension backend.
Permissions	Extension runs only on exact MITS domains, only after user action; no cookies/webRequest/all-sites permission. Data collection scope is visible and documented.
Moodle	A valid supported credential/feed imports a known sanitized fixture; malformed/expired/wrong-site feed fails closed and does not overwrite good data.
AMS	Only after approved source path exists: imports one known user's sanitized class-attendance fixture, maps Section B/B3 explicitly, and never imports roster/instructor-only fields.
Deltas	First scan classifies all in-scope stable refs as new; second identical scan produces zero new/changed; a one-field due-date/status change produces one changed item; same payload replay creates no duplicate report.
Partial coverage	If course/page listing is incomplete, output PARTIAL with coverage and do not create removal/deletion events.
Dashboard	Shows last successful scan separately from last attempted scan; links new/changed entries; no 'no changes' banner on errors. Source and timestamp are visible.
Preview data	No demo attendance/feed is shown as live. User can tell sample values from scanned data at a glance.
Disconnect/revoke	User can remove encrypted secret and revoke/reset source link; existing imported records follow an explicit retention choice.

Verification already done for this code snapshot

- ``node scripts/sync-frontend.mjs`` completed; maintained static UI is copied to `apps/web/public`.
- ``node --check js/app.js`` passed.
- ``pnpm build`` passed, including Next production compilation and TypeScript phase.
- No automated tests were added or run for the sign-in prompt. The prompt was not browser-smoke-tested after change.
- At the time of diagnosis, a local Next start-server process owned port 3000 and localhost requests returned HTTP 200. Unauthenticated GET `/api/v1/integrations` returned 401, as expected.

10. Relevant files and API inventory

Path from canonical root	What it contains / why inspect it
HANDOFF.md	Latest implementation notes and external setup gates. Read the newest section first; older sections contain historical states.
docs/FEASIBILITY.md	Portal evidence and unknowns. Reconcile owner-reported Google login with unverified actual SSO configuration.
docs/PROGRESS.md	Chronological implementation status; older notes may predate current applied migrations.
docs/SPEC.md	Long-term architecture plan. Browser Login Helper and scan/sync ideas are specification, not current shipped code.
js/app.js	Maintained client shell, integration prompt, #more page, status chips, demo feed/attendance sources, sign-in/status fetch.
index.html	Static dashboard shell and integration prompt markup.
css/v2.css	Current visual system and prompt/rail styles.
apps/web/public/	Deployable frontend copies. Run scripts/sync-frontend.mjs after root static changes.
apps/web/app/api/v1/integrations/route.ts	Signed-in user integration status GET.
apps/web/app/api/v1/integrations/moodle/route.ts	Moodle web-service token validation, encryption, disconnect. No ICS importer or scan route.
apps/web/app/api/v1/feed/route.ts	User-scoped feed read endpoint. Current static dashboard JS does not consume it.
apps/web/app/api/v1/events/route.ts	User-scoped manual event create/list APIs and event response fields.
packages/db/src/schema.ts	integrations, events/sourceRef unique, eventHistory, eventSources, attendanceLog, feedItems schemas.
packages/db/drizzle/0022_cuddly_reavers.sql	Moodle encrypted integration secret columns; migration reportedly applied to configured DB.
apps/web/lib/integration-crypto.ts	AES-256-GCM and HKDF purpose-specific secret encryption/decryption helpers.
scripts/sync-frontend.mjs	Copies root index.html/css/js into apps/web/public.
.env.example	Safe variable names. Never print .env values; secrets exist only in local private .env/host settings.

Existing vs. missing API

Capability	Status
GET /api/v1/integrations	Exists; authenticated status.
POST /api/v1/integrations/moodle	Exists; verify and save web-service token only.
DELETE /api/v1/integrations/moodle	Exists; disconnect Moodle.
GET /api/v1/feed	Exists; feed storage endpoint, not currently consumed by Radar UI.
POST pairing-code / worker-public-key / integration session	Described in docs/SPEC.md only; verify route existence before reuse. Not found in current implementation search.
POST manual scan / scan history / report	Missing.
Moodle calendar-feed save/scan	Missing.
AMS read-only import	Missing; source path not yet approved/verified.

11. Owner actions and unresolved choices

The user's other AI should separate code work from account/admin work. Do not ask the owner to paste secrets into chat. If an action requires a private token, feed URL, or credential, make a secure in-app field first and instruct the owner to enter it there.

Owner decisions or evidence needed

- 1 Confirm which Google identity the MITS portals use and whether the login screens show a Google SSO button for the same account. This confirms only the portal login convenience, not data authorization.
- 2 For Moodle, ask MITS support to enable a limited read-only Web Services service/token for needed functions, OR after the importer exists generate a dynamic calendar feed and enter it inside UniDash. Never share that feed URL in chat or commit it.
- 3 For AMS, ask the institute for supported student read-only API/CSV/export and whether a local read-only browser extension is permitted. Supply only sanitized screenshots/DOM examples without account IDs, cookies, tokens, classmates, or marks not required for the feature.
- 4 Confirm first scan scope: Moodle events/assignments only versus resources/submissions; AMS attendance only versus timetable/exam notices. Narrow scope reduces privacy and parser risks.
- 5 After the helper is built, install the unpacked extension locally in Edge and perform a manual smoke scan while signed into the user's own accounts.

Not required for current manual scan version

- No hourly scheduler or 24/7 worker polling.
- No Gmail password entry, Google password storage, browser cookie export, or session reuse by the server.
- No app-wide Google Drive scope; Drive storage is separate and already implemented.
- No account-wide/full-page extraction if user-owned tables can be read narrowly.
- No assumption that an empty scan means no updates; show coverage, source connection, and parser status.

Current extension recommendation

Build a local Edge-compatible Manifest V3 extension with exact Moodle/AMS origins and user-click-only execution. To send data to UniDash, use a tightly scoped pairing flow or an exact-origin extension message bridge. Avoid use of cookies APIs. Extension should package no portal credentials. Keep the first delivery unpacked/local, because production origin, store publishing, and institutional approval are not selected.

12. Specialized AI continuation prompt

Copy the prompt below into the other AI after attaching this PDF and giving it access to the project folder. This report is a dated snapshot; verify current source.

Continue UniDash Moodle and AMS integration using this report and repository. Begin at the nested canonical project root and read HANDOFF.md, docs/FEASIBILITY.md, docs/PROGRESS.md, then inspect current source before changing it. Goal: user signs into Moodle/AMS in Edge, clicks Scan now, and sees new or changed records since the previous successful scan in a dashboard report. Hourly/background scans are out of scope. UniDash Google sign-in is not portal authorization; use each portal's own SSO if MITS supports it. Build a narrow Manifest V3 Edge extension with exact MITS hosts and click-only visible-page reading. Do not access cookies, passwords, or session IDs. Add secure pairing and validated ingestion. Prefer an approved read-only Moodle API or protected calendar feed. AMS has no verified API/export; do not fabricate attendance or claim sync before an approved source or safe visible-page adapter is verified. The integration prompt currently opens portals but does not scan. Dashboard feed and attendance still use demo data. Reuse existing feed API and event/source-history schema where appropriate. Add scan-run persistence, stable source references, hashes, deduplicated feed/history, partial-scan safeguards, and an accurate dashboard report. Keep secrets out of files, chat, and logs. Never generate/reveal a private Moodle feed URL on the owner's behalf. Do not add or run tests unless asked; routine build/type/lint checks are acceptable. Make progress independently, state exactly what needs owner/MITS-admin action, update HANDOFF.md, and verify claims in source.

13. References and provenance

Project facts in this report come from the current source tree, HANDOFF.md, docs/FEASIBILITY.md, docs/PROGRESS.md, and the owner's statements in this conversation. Portal observations are read-only inspections already summarized in project notes; this PDF includes no passwords, tokens, feed URL, cookie, session ID, exact attendance values, or other students' information.

Product/standard references below are official documentation. Generic Moodle documentation explains capabilities and setup patterns but does not prove that MITS has enabled the same feature or the exact API functions needed.

[Moodle: Using web services \(Moodle 5.2 documentation\)](#)

Setup requirements, tokens, service/function selection, and capability notes.

[Moodle: Security keys \(Moodle 5.2 documentation\)](#)

User token availability depends on moodle/webservice:createtoken.

[Moodle: Calendar export \(MoodleDocs\)](#)

Export calendar/ICS URL concept; MITS instance remains site-specific.

[Chrome for Developers: Content scripts](#)

Content script isolation and site interaction.

[Chrome for Developers: Declare permissions](#)

Host permission model and permission warnings.

[Chrome for Developers: externally_connectable](#)

Restrict which web pages may message an extension.

End of handoff. The most important truth for the next AI: the connection prompt and external portal launch now exist; automatic sign-in and the scan/delta/dashboard-report pipeline do not.